# Lecture 4.1

# Image Transforms

Instructor Name:

**Professor Dr. Jahangir Alam**

**Dept. of CSE, BAUST**

# Image Transforms

**Image Transforms** are mathematical operations used in image processing to convert an image from one domain (usually spatial domain) to another (often frequency domain). This helps in tasks like compression, enhancement, filtering, and feature extraction.

An image transform changes how image information is represented.

- **Spatial domain** → pixels (intensity values)
- **Transform domain** → frequency or other representations

Instead of working directly on pixel values, we analyze patterns like edges, textures, and frequencies.

# Important Image Transforms

| | |
|---|---|
| **Fourier Transform (FT)** | ✓ **Low frequencies → smooth areas**<br>✓ **High frequencies → edges & details**<br>✓ **Image filtering (blur, sharpen)**<br>✓ **Compression (JPEG partially relies on frequency ideas)** |
| Discrete Cosine Transform (DCT) | ✓ Widely used in **image compression**<br>✓ Core of JPEG compression<br>✓ Most energy is concentrated in a few coefficients → reduces data |
| Wavelet Transform | ✓ Gives both **frequency + spatial location**<br>✓ Used in modern compression (JPEG2000)<br>✓ Better for analyzing local features |
| Hadamard Transform | ✓ Fast computation (no multiplication)<br>✓ Used in pattern recognition |

# Why Image Transforms

- ✓ Remove noise (filtering in frequency domain)
- ✓ Compress images efficiently
- ✓ Detect edges and features
- ✓ Enhance image quality

Original Image → Apply Transform → Modify Coefficients → Inverse Transform → Enhanced Image

# Spatial Operations

(1) Single-pixel operations,

(2) Neighborhood operations, and

(3) Geometric spatial transformations.

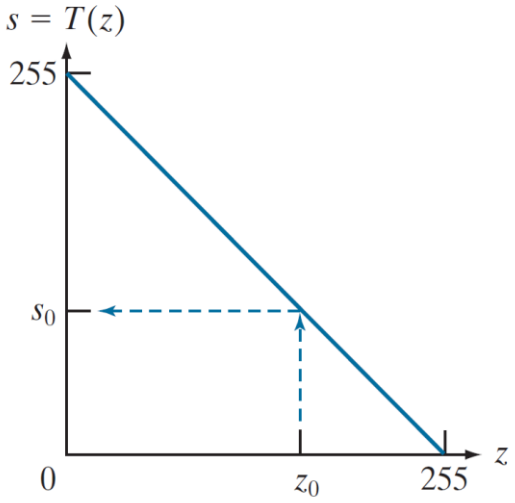
# 1. Single-Pixel Operations

$s = T(z)$

$s$ is the (mapped) intensity of the corresponding pixel in the processed image.

Intensity

Transformation function




transformation used to obtain the *negative* (sometimes called the *complement*) of an 8-bit image.

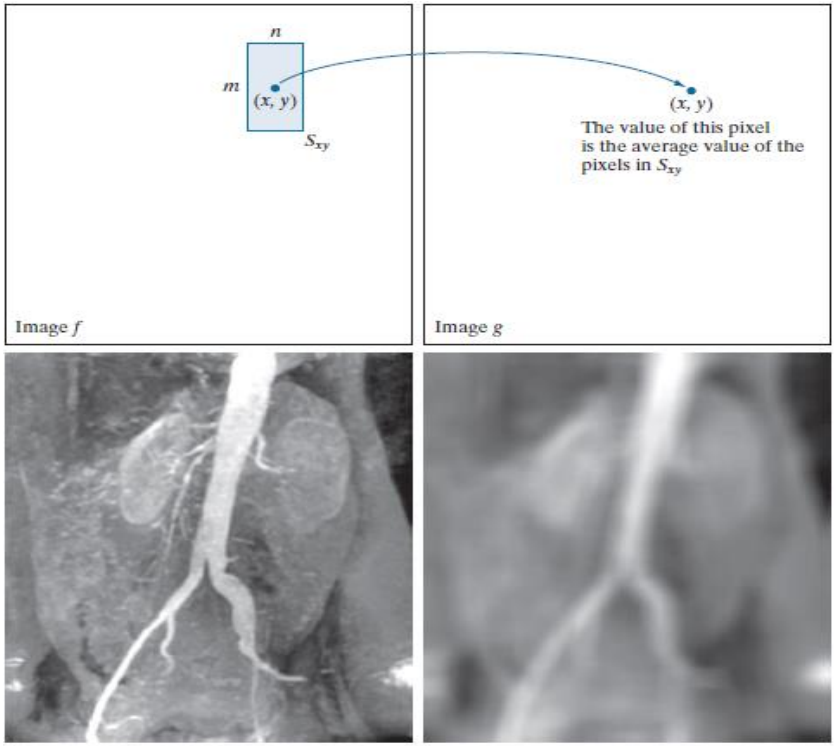
# 2. Neighborhood Operations

Row and column coordinates of the pixels

$$g(x,y) = \frac{1}{mn} \sum_{(r,c) \in S_{xy}} f(r,c)$$

neighborhood of pixels

set of coordinates of a neighborhood centered on an arbitrary point ($x$, $y$) in an image, $f$.



# 3. Geometric Transformations

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \mathbf{T} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} t_{11} & t_{12} \\ t_{21} & t_{22} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \mathbf{A} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

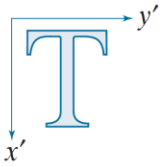
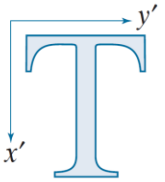
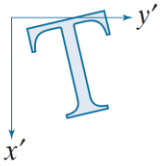
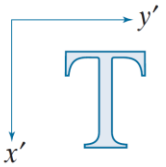
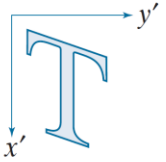
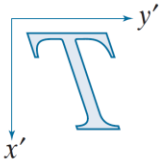
**1.** Spatial transformation of coordinates.

**2.** Intensity interpolation that assigns intensity values to the spatially transformed pixels.

*Scale*, *rotate*, *translate*, or *sheer* an image;

Intensity interpolation;

# Affine Transformations

| Transformation Name | Affine Matrix, A | Coordinate Equations | Example |
|---|---|---|---|
| Identity | $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$ | $x' = x$<br>$y' = y$ |  |
| Scaling/Reflection (For reflection, set one scaling factor to –1 and the other to 0) | $\begin{bmatrix} c_x & 0 & 0 \\ 0 & c_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$ | $x' = c_x x$<br>$y' = c_y y$ |  |
| Rotation (about the origin) | $\begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$ | $x' = x\cos\theta - y\sin\theta$<br>$y' = x\sin\theta + y\cos\theta$ |  |
| Translation | $\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$ | $x' = x + t_x$<br>$y' = y + t_y$ |  |
| Shear (vertical) | $\begin{bmatrix} 1 & s_v & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$ | $x' = x + s_v y$<br>$y' = y$ |  |
| Shear (horizontal) | $\begin{bmatrix} 1 & 0 & 0 \\ s_h & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$ | $x' = x$<br>$y' = s_h x + y$ |  |

>>help imcrop
>> help affine2d
>>help imresize
>>help imrotate
>> help imtranslate
>> help impyramid

# Geometric Transformations (2D and 3D Scaling, Translation, Rotation, Affine Transform)

Given

A simple 2D point and 3D point to transform:

- 2D point $P$=(2,3)
- 3D point $Q$=(1,2,3)

**1. 2D Transformations**

**a) Scaling**

Scale factors: $Sx$=2, $Sy$=3

$$P_{scaled} = (x \times Sx, y \times Sy) = (2 \times 2, 3 \times 3) = (4, 9)$$

**b) Translation**

Translation vector: $T = (tx, ty) = (5, -2)$

$$P_{translated} = (x + tx, y + ty) = (2 + 5, 3 - 2) = (7, 1)$$

# Geometric Transformations (2D and 3D Scaling, Translation, Rotation, Affine Transform)

**c) Rotation**

Rotate point $P$ about origin by $\theta = 90^o$ counterclockwise.

Rotation matrix:

$$R = \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

Multiply:

$$P_{rotated} = R \times \begin{bmatrix} 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix} = \begin{bmatrix} -3 \\ 2 \end{bmatrix}$$

# Geometric Transformations (2D and 3D Scaling, Translation, Rotation, Affine Transform)

**d) Affine Transformation**

General affine transform formula:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Choose parameters:

$$a = 1, b = 2, c = 3, d = 0, e = 1, f = -1$$

Calculate:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 11 \\ 2 \\ 1 \end{bmatrix}$$

Thus,

$$P_{affine} = (11, 2)$$

# Geometric Transformations (2D and 3D Scaling, Translation, Rotation, Affine Transform)

## 2. 3D Transformations

### a) Scaling

Scale factors: $Sx = 2, Sy = 3, Sz = 4$

$$Q_{scaled} = (1 \times 2, 2 \times 3, 3 \times 4) = (2, 6, 12)$$

### b) Translation

Translation vector: $T = (tx, ty, tz) = (5, -2, 3)$

$$Q_{translated} = (1 + 5, 2 - 2, 3 + 3) = (6, 0, 6)$$

# Geometric Transformations (2D and 3D Scaling, Translation, Rotation, Affine Transform)

**c) Rotation**

Rotate point $Q$ around z-axis by $\theta = 90^o$ CCW.

Rotation matrix about z-axis:

$$R_z = \begin{bmatrix} cos\theta & -sin\theta & 0 \\ sin\theta & cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Multiply:

$$Q_{rotated} = R_z \times \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} -2 \\ 1 \\ 3 \end{bmatrix}$$

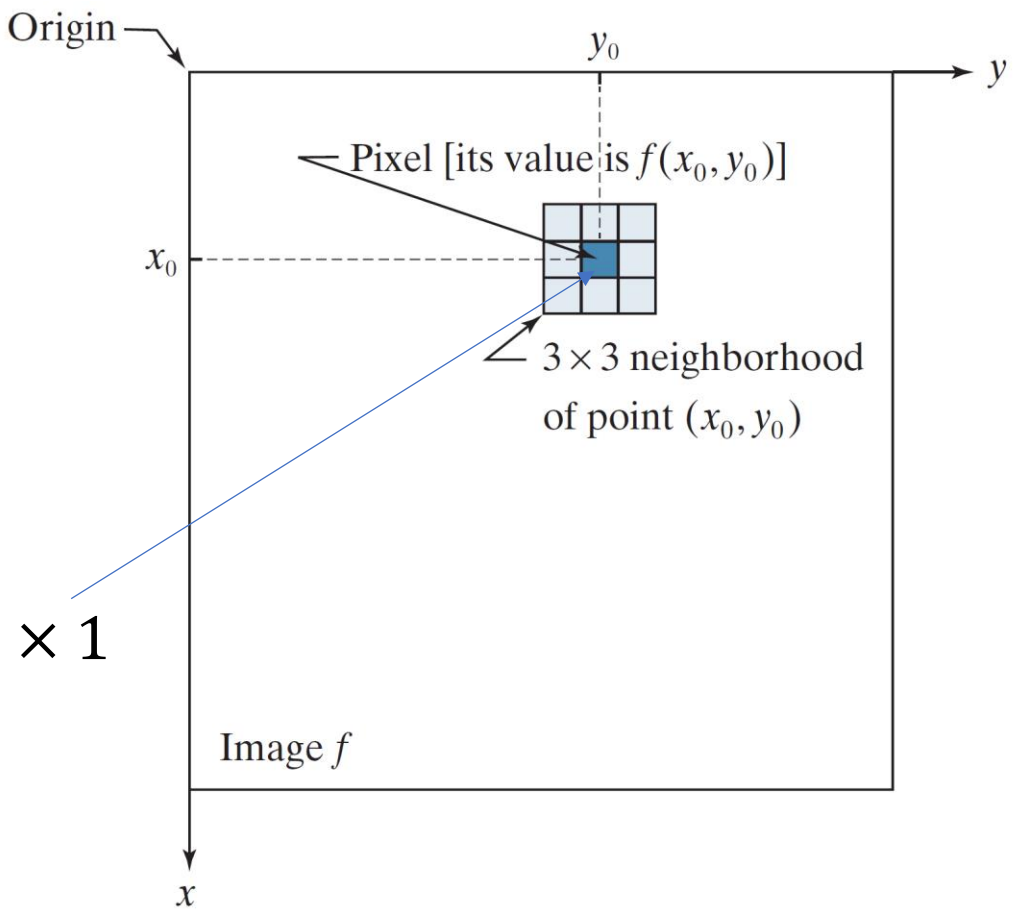
| Transformation | Input | Output |
|---|---|---|
| 2D Scaling | (2,3) | (4,9) |
| 2D Translation | (2,3) | (7,1) |
| 2D Rotation (90° CCW) | (2,3) | (-3,2) |
| 2D Affine | (2,3) | (11,2) |
| 3D Scaling | (1,2,3) | (2,6,12) |
| 3D Translation | (1,2,3) | (6,0,6) |
| 3D Rotation (Z-axis) | (1,2,3) | (-2,1,3) |

# Intensity Transformations & Spatial Filtering

$$g(x,y) = T[f(x,y)]$$

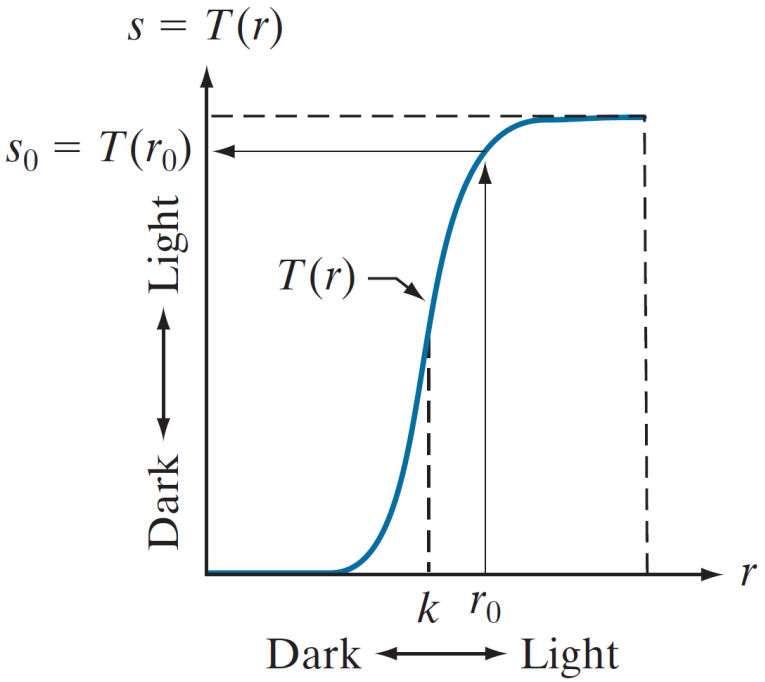
Neighborhood

Smallest possible neighborhood is of size: $1 \times 1$



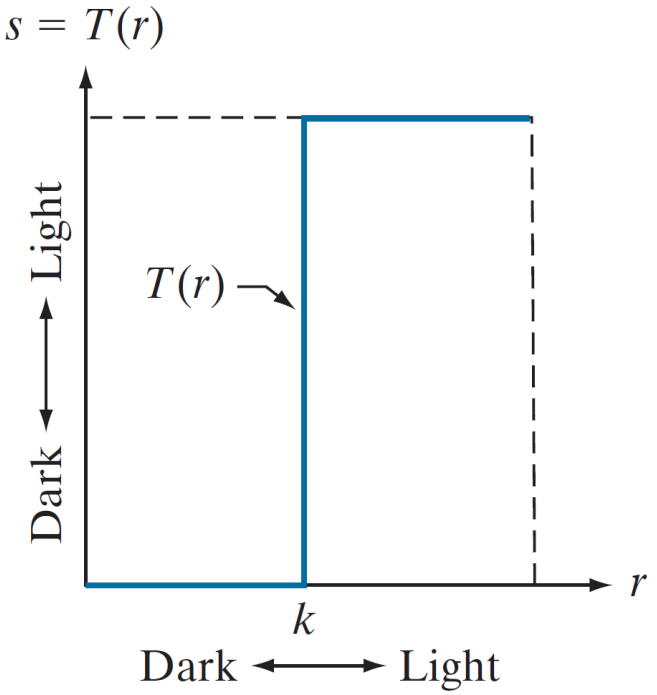
# Intensity Transformations & Spatial Filtering

$s = T(r)$




Intensity (*gray-level,* or *mapping*) *transformation function*

*contrast stretching depends on k*

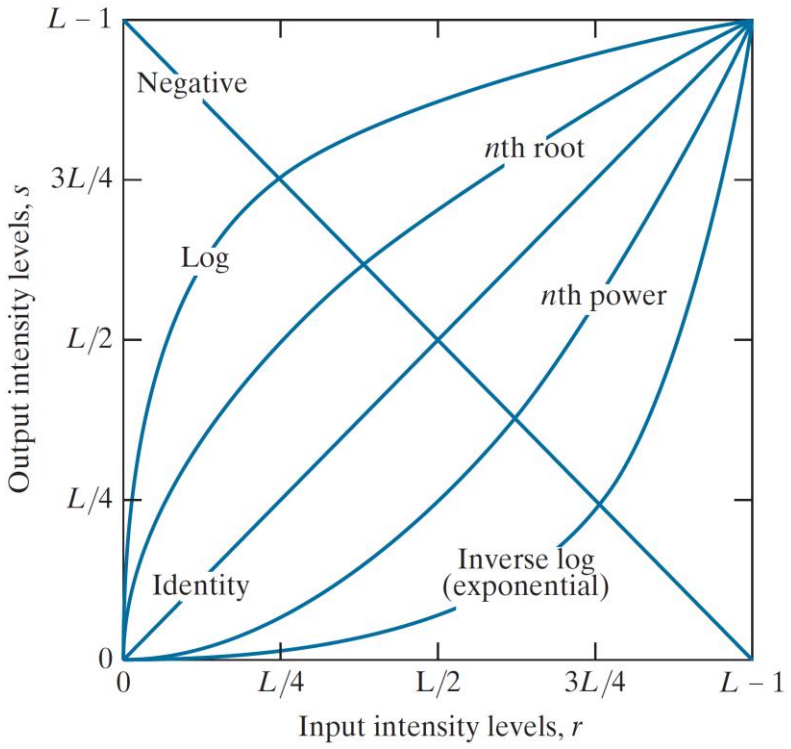


# Intensity Transformations Functions

## Intensity Transformations Functions

- ✓ Contrast Stretching
- ✓ Logarithmic Transform
- ✓ Power-Law (Gamma) Transform
- ✓ Histogram Equalization (enhance contrast).

- linear (negative & identity transformations),
- logarithmic (log and inverse-log transformations), and
- power-law (*n*th power and *n*th root transformations)



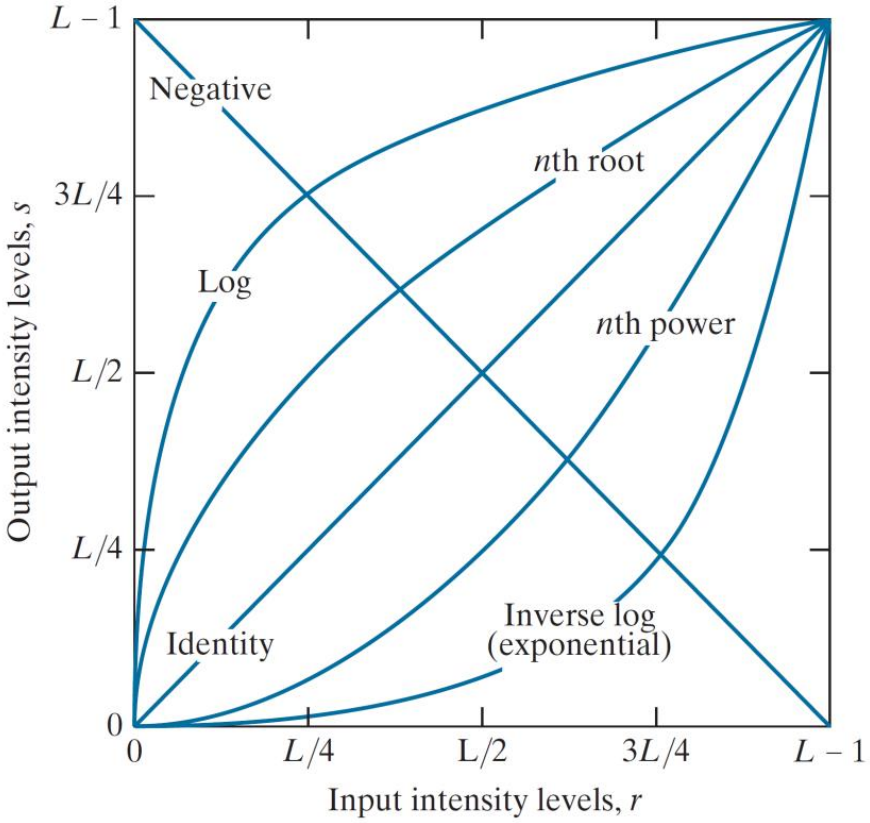
# Intensity Transformations Functions

Image Negatives:

[0, L-1]

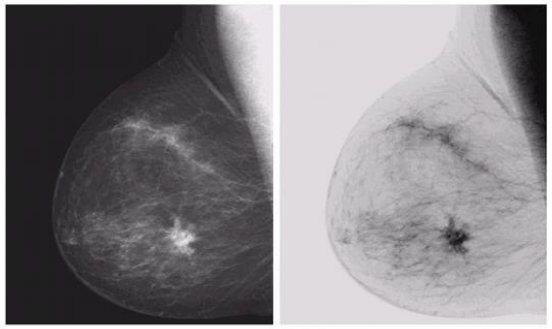
s=L-1-r

- Linear (negative and identity transformations),



# Image Negatives

- **Algorithm**
- 1. Read RGB color image into the MATLAB environment using Matlab inbuilt function imread()
- 2. Calculate the levels of the image, for example an 8-bit image has 256 levels
- 3. Use the formula stated above on every pixel of the image to get corresponding negative pixel value.
- 4. Convert each RGB pixel value at location (i, j) to its negative image values and assign it to the corresponding location (i, j) of another matrix
- 5. Display the negative image using Matlab in-built imshow() function.



**(left) Original digital mammogram. (right) Negative image obtained using the negative transformation**

# Intensity Transformations Functions

Log Transformations:

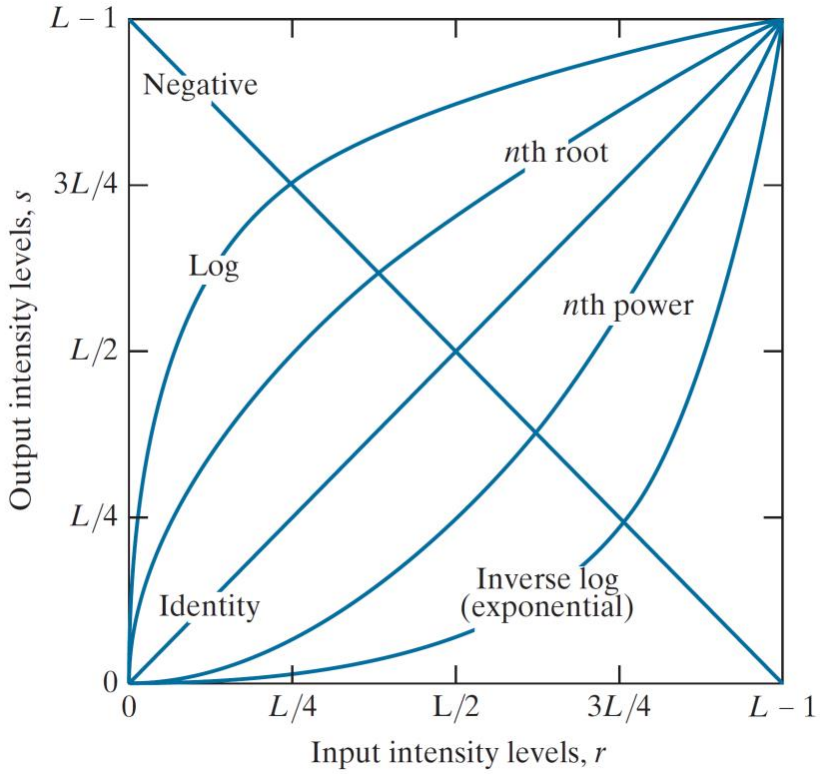
- logarithmic (log and inverse-log transformations)

$$s = c\ log(1 + r)$$

constant

$$r \geq 0$$

[0, L-1] ⟹ [0, L/4] ⟺ [0, 3L/4]

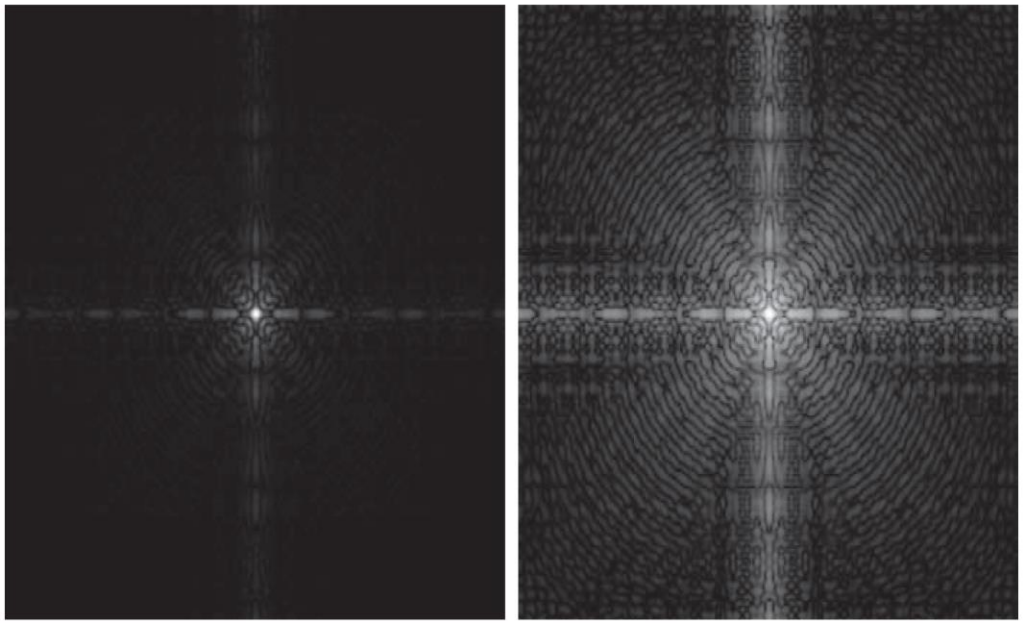


# Intensity Transformations Functions

(a) Fourier spectrum displayed as a grayscale image.

(b) Result of applying the log transformation log-transformation with $c = 1$.

Both images are scaled to the range [0, 255].



# Intensity Transformations Functions

Power-Law (Gamma) Transformation:

[0, L-1]

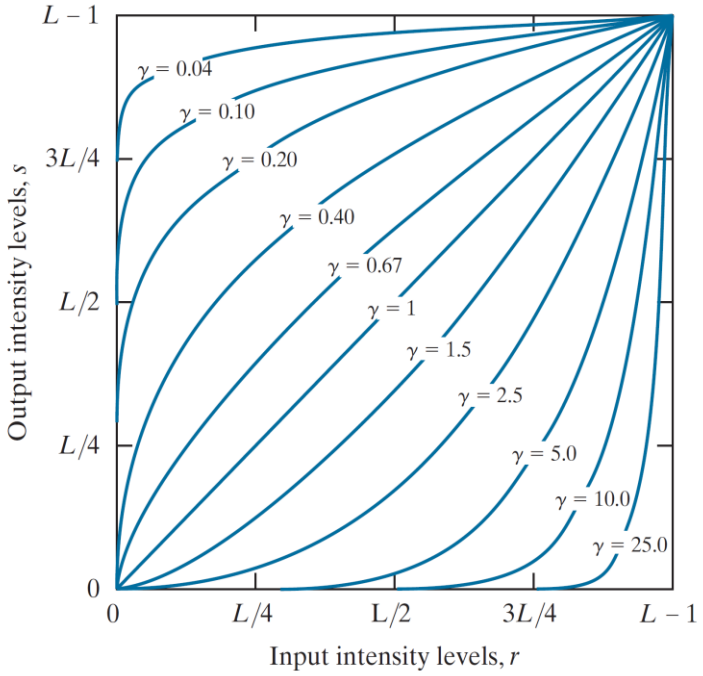
- power-law (*n*th power and *n*th root transformations)

$s = c\, r^{\gamma}$ Or, $s = c\,(1+\varepsilon)^{\gamma}$

$\gamma > 1$ Light

$\gamma < 1$ Dark

$c = \gamma = 1$ Identity transformation




Grey levels must be in the range [0.0, 1.0]

# Intensity Transformations Functions

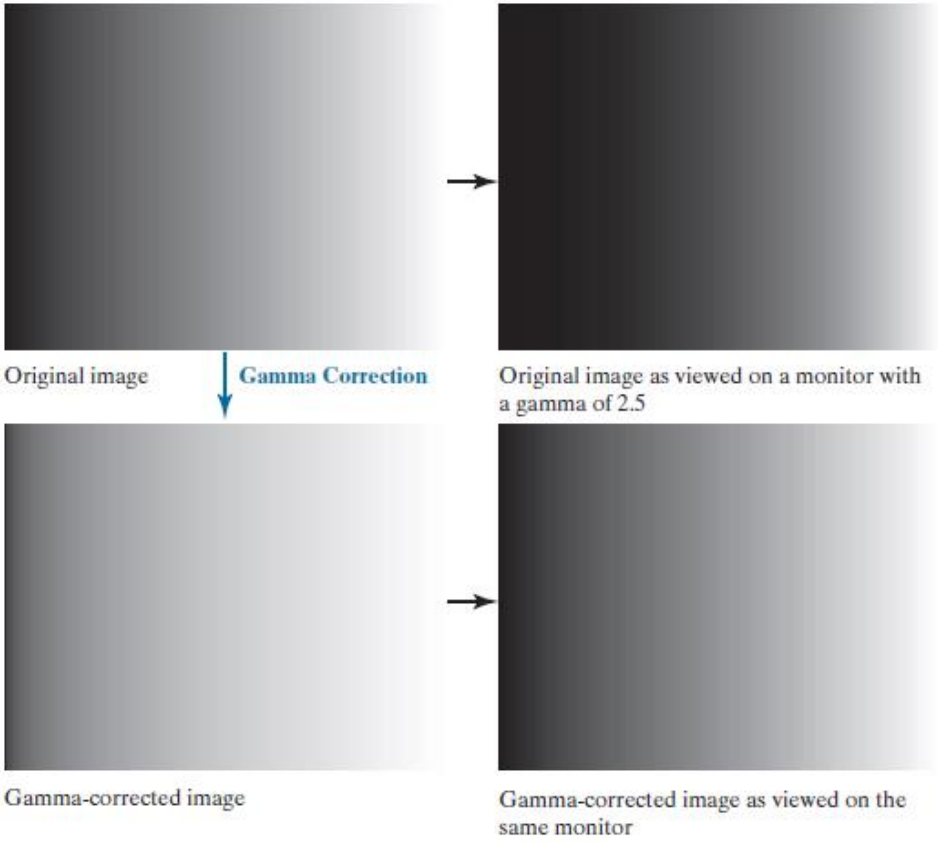
*Gamma correction* or *gamma encoding*

$$s = c\, r^{\gamma}$$

cathode ray tube (CRT) devices have an intensity-to-voltage response that is a power function, with exponents varying from approximately 1.8 to 2.5.

As the curve for $\gamma = 2.5$

The picture will become darker.

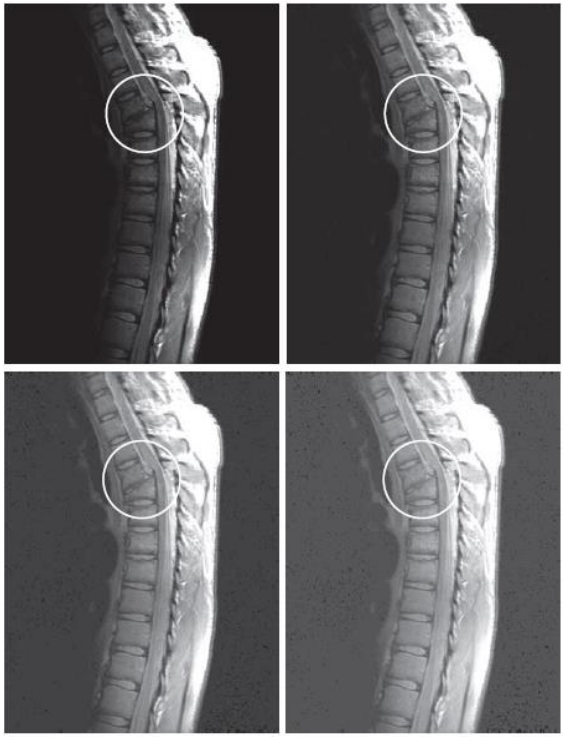


# Intensity Transformations Functions

$$s = c\, r^{\gamma}$$

**Contrast enhancement using power-law intensity transformations.**

(a) Magnetic resonance image (MRI) of a fractured human spine (the region of the fracture is enclosed by the circle).



(b)–(d) Results of applying the power-law (Gamma) transformation with $\boldsymbol{c = 1}$ **and** $\boldsymbol{\gamma =}$ **0.6, 0.4, & 0.3**, respectively.

# Intensity Transformations Functions

(a) Aerial image.

(b)–(d) Results of applying the power-law (Gamma) transformation with $\gamma$ = 3.0, 4.0, and 5.0, respectively.

($c$ = 1 in all cases.)



## Effect of Decreasing Gamma

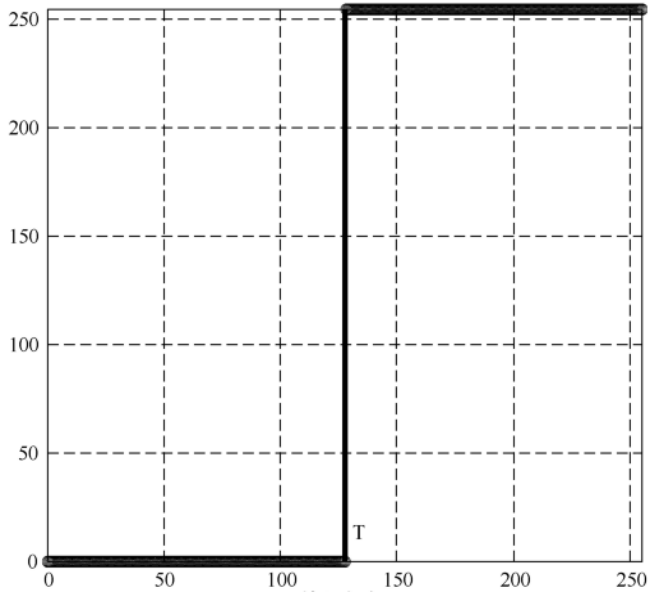
When the **i** is reduced too much, the image begins to reduce contrast to the point where the image started to have very slight **wash-out** look, especially in the background.

# Piecewise Linear Transformations Functions

1. Grayscale Threshold Transform (Gray-level Slicing)
2. Contrast Stretching
3. Bit-plane Slicing

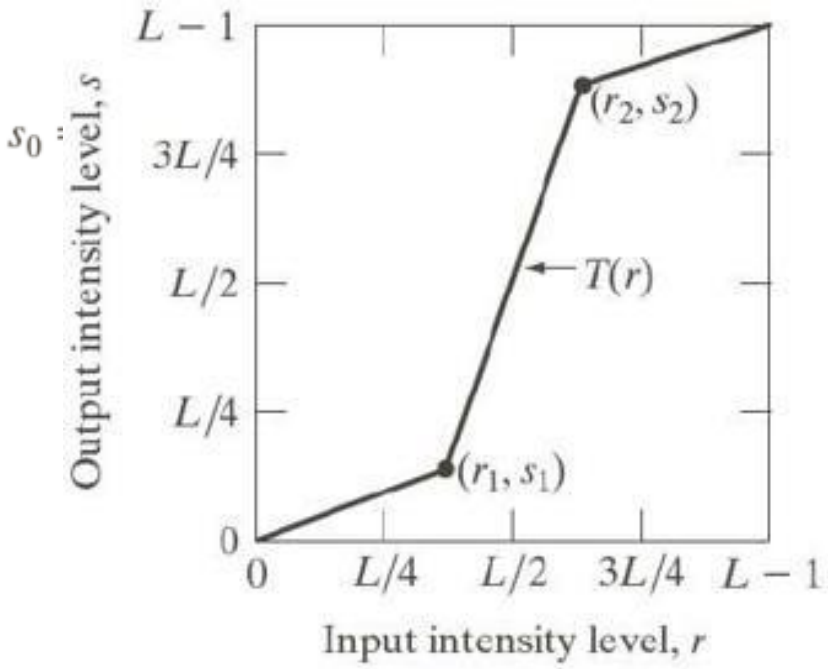
# 1. Grayscale Threshold Transform

- converts a grayscale image into a black and white binary image.
- dividing line.
- If the gray value of a pixel is smaller than the dividing, the intensity of the pixel is set to 0, otherwise it's set to 255.
- The value of the dividing line is called the threshold. The grayscale threshold transform is often referred to as thresholding, or binarization.



# 2. Contrast-Stretching Transformation

Goal: enhances the gray contrast between different parts of an image, areas of interest, and suppresses the gray contrast for areas that are not of interest.




## 1. Power-Law Contrast-Stretching

If T(r) is the effect of applying the transformation to every pixel to generate the corresponding pixels produce higher contrast than the original image, by:

- Darkening the levels <k in the original image
- Brightening the levels >k in the original image

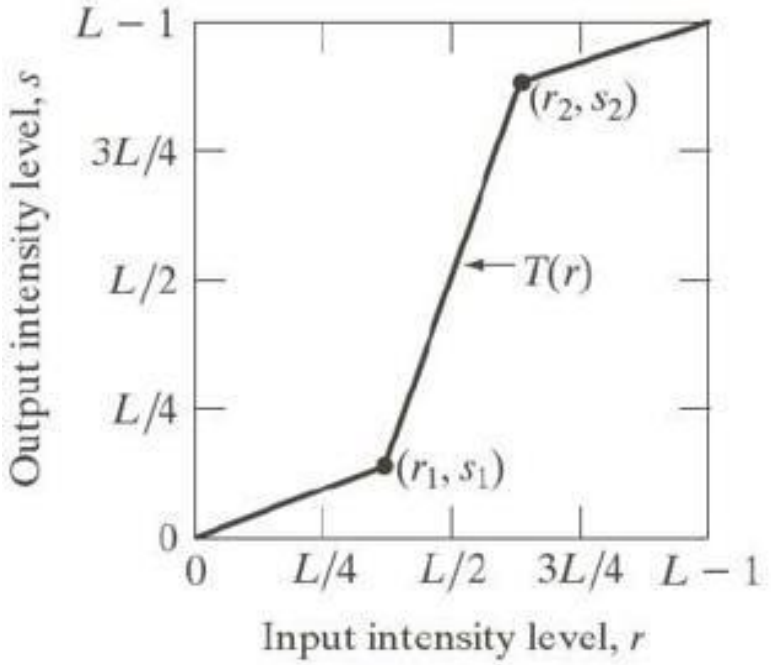
# 2. Contrast-Stretching Transformation

**2. Linear Contrast-Stretching Transformation**

Points (r1, s1) and (r2, s2) control the shape of the transformation.

The selection of control points depends upon the types of image and varies from one image to another image.

If r1 = s1 and r2 = s2 then the transformation is linear and this doesn't affect the image.




- For 0 <= x <= r1 output = s1 / r1 * x
- For r1 < x <= r2 output = ((s2 – s1)/(r2 – r1))*(x – r1) + s1
- For r2 < x <= L-1 output = ((L-1 – s2)/(L-1 – r2))*(x – r2) + s2

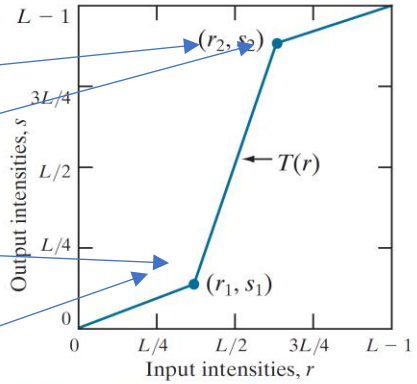
# 3. Piecewise Linear Transformations Functions

## Contrast Stretching

maximum intensity levels

control the shape of the transformation function

minimum intensity levels









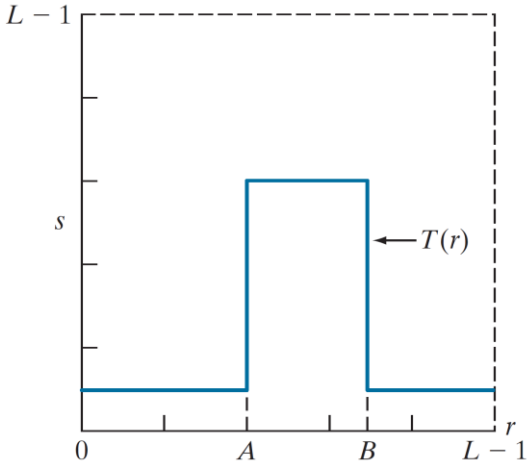

# 3. Piecewise Linear Transformations Functions

## Intensity-Level Slicing

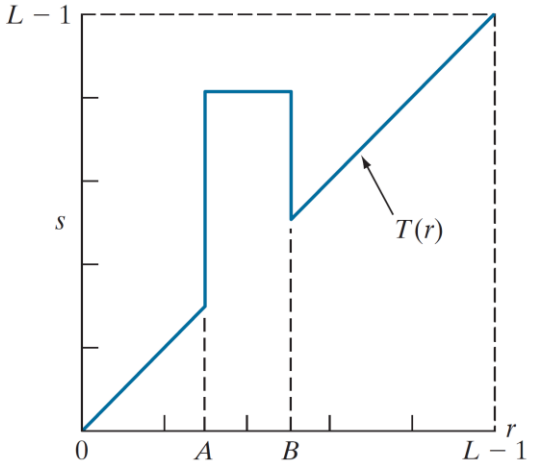
[0, L-1]

(r1,s1)=(m, 0) and

(r2,s2)=(m, L-1)




Binary image




Brighten/darken intensity range

# 4. Bit-plane Slicing

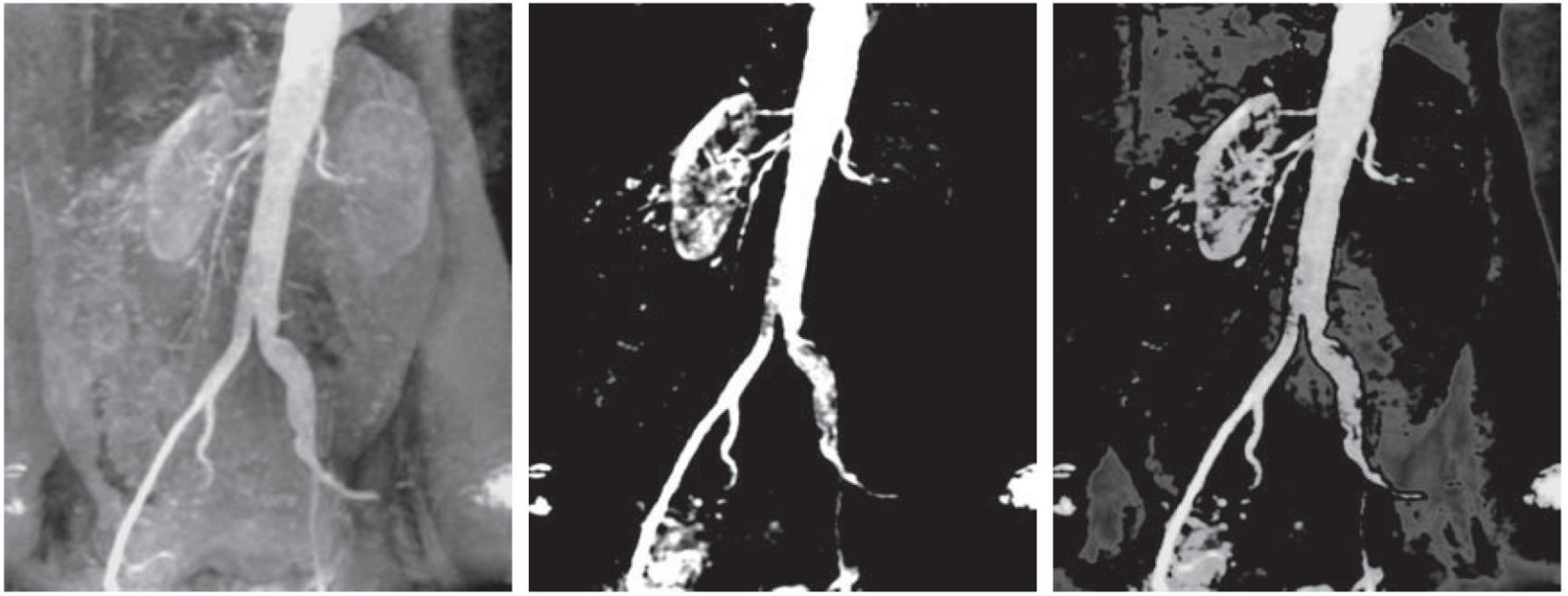
- Each pixel of a grayscale image is stored as an 8-bit byte.
- The intensity spans from 0 to 255, which is ‘00000000’ to ‘11111111’ in binary.
- Bit-plane slicing refers to the process of slicing a matrix of 8 bits into 8 planes.
- MSB (the 8th bit from right to left) carries the most weight.
- LSB (the 1st bit from right to left) carries the least weight.

# Example: Intensity-Level Slicing

Aortic angiogram

Gray intensities
slicing transformation

selected range set
near black



# 1-D Impulse and Shifting Properties

$$\delta(t) = \begin{cases} \infty & \text{if } t = 0 \\ 0 & \text{if } t \neq 0 \end{cases}$$

$$\int_{-\infty}^{\infty} \delta(t)\,dt = 1$$

$$\int_{-\infty}^{\infty} f(t)\,\delta(t)\,dt = f(0)$$

$$\int_{-\infty}^{\infty} f(t)\,\delta(t - t_0)\,dt = f(t_0)$$

# 2-D Impulse and Sifting Properties

$$\delta(x, y) = \begin{cases} 1 & \text{if } x = y = 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\sum_{x=-\infty}^{\infty} \sum_{y=-\infty}^{\infty} f(x, y)\, \delta(x, y) = f(0, 0)$$

$$\sum_{x=-\infty}^{\infty} \sum_{y=-\infty}^{\infty} f(x, y)\, \delta(x - x_0, y - y_0) = f(x_0, y_0)$$

# Nyquist–Shannon Sampling Theorem

Sampling rate must be at least twice as high as the highest frequency you want to represent.

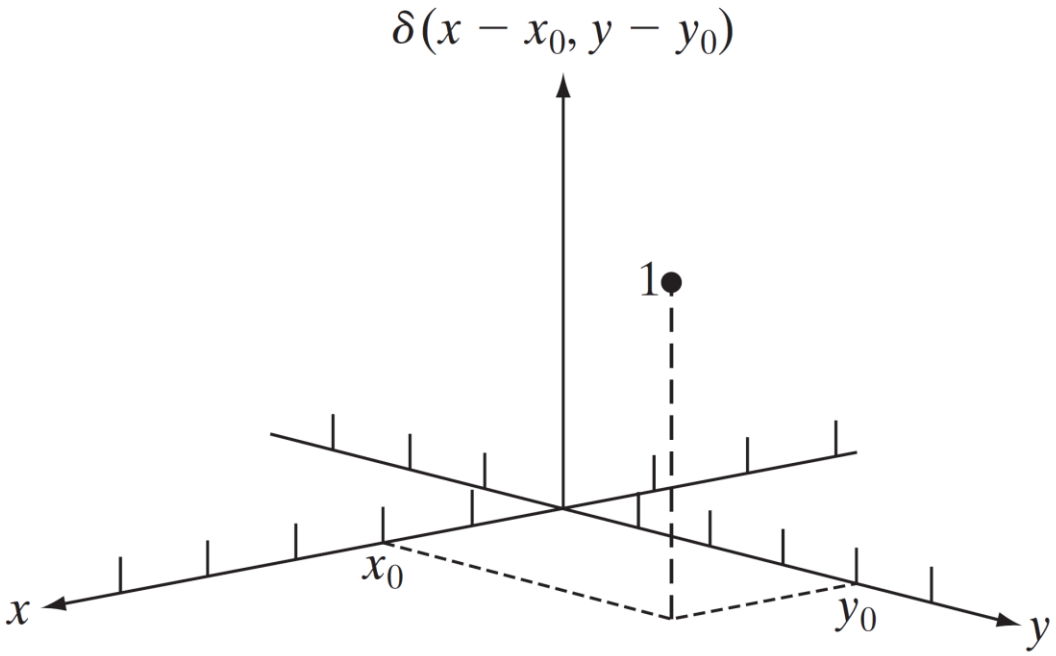
The sampling theorem guarantees that an analogue signal can be perfectly recovered as long as the sampling rate is at least twice as large as the highest-frequency component of the analogue signal to be sampled.

$$F_s \geq 2F_{\max}$$

If the highest frequency contained in an analog signal $x_a(t)$ is $F_{max}$. And the signal is sampled at a rate $F_s >= 2F_{max}$, then $x_a(t)$ can be exactly recovered from its sample values.

**The sampling rate, $F_S >= 2F_{max}$ is called the Nyquist rate.**

# 2-D Impulse and Sifting Properties



# Thank You

# Lecture 4.2

## Image Transforms

## Fourier Series

Instructor Name:
**Professor Dr. Jahangir Alam**
**Dept. of CSE, BAUST**

# Fourier Series

The Fourier Series represents a periodic function (signal/Image Signal) as a sum of simple sine & cosine waves.

It is useful for compression, filtering, edge detection, noise removal, and analysis.

$$f(t) = \sum_{n=-\infty}^{\infty} c_n e^{j\frac{2\pi n}{T}t}$$

$$c_n = \frac{1}{T}\int_{-T/2}^{T/2} f(t) e^{-j\frac{2\pi n}{T}t} dt \qquad \text{for } n = 0, \pm 1, \pm 2, \ldots$$

$$f(t) = a_0 + \sum_{n=1}^{\infty}\left[a_0 cos\left(\frac{2\pi nt}{T}\right) + b_n sin\left(\frac{2\pi nt}{T}\right)\right]$$

DC term (Average value):

$$a_0 = \int_0^T f(t)\, dt$$

# 2D Fourier Series

- For a grayscale input image $f(x, y)$ with period $M$ in $x$-direction and $N$ in $y$-direction:

$$f(x,y) = \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u,v) e^{j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$$

- It is a 2D Fourier Transform, and generalized Fourier Series for discrete signals (like digital images).

**Interpretation**

- Low frequencies (small $u, v$) → capture smooth variations (overall brightness, background).
- High frequencies (large $u, v$) → capture edges, textures, fine details.
- Fourier Series breaks an image into smooth part + details.

# Why Fourier in CV/Images?

*Fourier Series in image processing = represent an image as a sum of sinusoidal patterns.*

- An image is just a 2D signal (intensity values across rows & columns).

- Just like a 1D periodic signal (sound wave) can be expressed using Fourier Series, an image (2D signal) can be expressed as a sum of 2D sinusoidal waves.

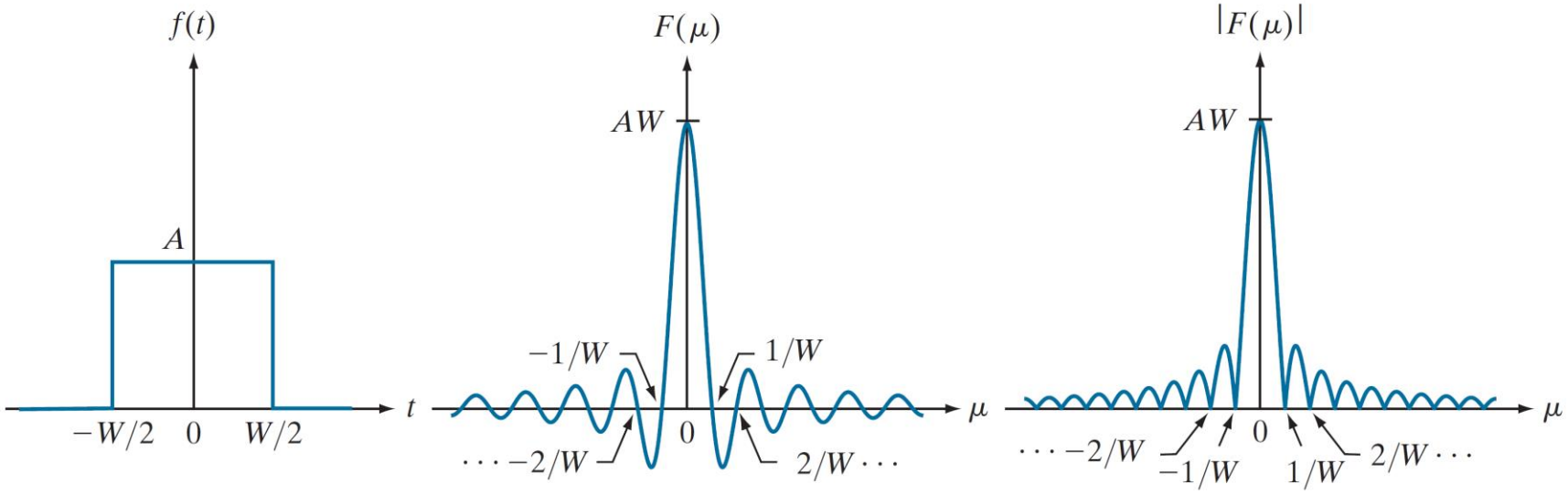
# Applications of Fourier Series

- **Signal Processing** (representing periodic signals)
- **Music & Audio compression** (Fourier for sound waves)
- **Image compression** (JPEG uses related Fourier/DCT ideas)
- ✓ Uses DCT Fourier-related transform.

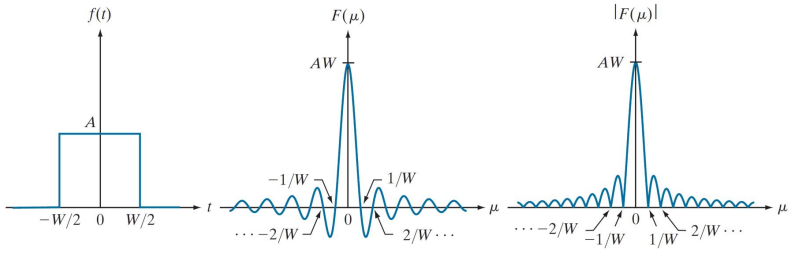
- **Image Filtering**
- Keeps low-frequency terms (important structure), discards high-frequency (less noticeable).
- ✓ Blurring: keep only low-frequency terms.
- ✓ Sharpening: enhance high-frequency terms.

- **Image Reconstruction**
- ✓ Approximate an image using a few Fourier coefficients (like a square wave).
- ML (Fourier features for kernels)
- DSP, Electrical engineering (AC circuits, harmonics)

# Fourier Spectrum



# Fourier Spectrum




$$F(\mu) = \int_{-\infty}^{\infty} f(t)e^{-j2\pi\mu t}dt = \int_{-W/2}^{W/2} Ae^{-j2\pi\mu t}dt$$

$$= \frac{-A}{j2\pi\mu}\left[e^{-j2\pi\mu t}\right]_{-W/2}^{W/2} = \frac{-A}{j2\pi\mu}\left[e^{-j\pi\mu W} - e^{j\pi\mu W}\right]$$

$$= \frac{A}{j2\pi\mu}\left[e^{j\pi\mu W} - e^{-j\pi\mu W}\right]$$

$$= AW\frac{\sin(\pi\mu W)}{(\pi\mu W)}$$

$$\text{sinc}(m) = \frac{\sin(\pi m)}{(\pi m)} \qquad |F(\mu)| = AW\left|\frac{\sin(\pi\mu W)}{(\pi\mu W)}\right|$$

# Image Transforms using DFT

$$F(u,v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y) r(x, y, r, v)$$

2-D *linear* Forward *transforms of f(x,y)*

Input image

Forward transformation kernel

$u = 0,1,2,\ldots,M-1$

$v = 0,1,2,\ldots,N-1$

# Image Transforms using DFT

$$f(x,y) = T(u,v)^{-1}$$

$$f(x,y) = \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} T(u,v)\, s(x,y,u,v)$$

Inverse transformation kernel

*Transform pair.*

$$T(u,v) = f(x,y)$$

$x = 0,1,2,\ldots,M-1$

$y = 0,1,2,\ldots,N-1$

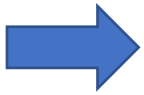
# Image Transforms using DFT

```
>>help fourier
>> help fft
>> Fourier transform
```

Forward transformation kernel is said to be *separable* if,

$$r(x, y, u, v) = r_1(x, u) r_2(y, v)$$

$$r(x, y, u, v) = r_1(x, u) r_1(y, v)$$



kernel is said to be *symmetric*

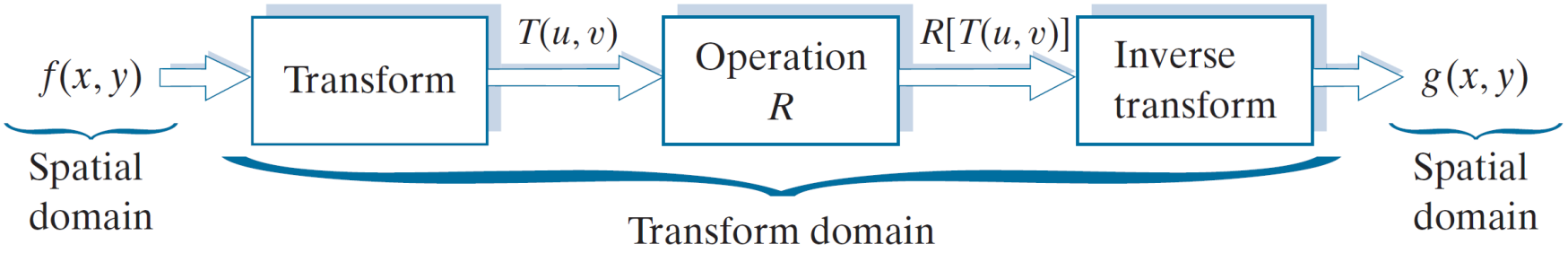
*Fourier transform* forward kernels:

$$r(x, y, u, v) = e^{-j2\pi(ux/M + vy/N)}$$

*Fourier transform* inverse kernels:

$$s(x, y, u, v) = \frac{1}{MN} e^{j2\pi(ux/M + vy/N)}$$

# Image Transforms using DFT




>>help fourier
>> help fft
>> Discrete Fourier transform

*DFT pair:*

$$T(u,v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y) e^{-j2\pi(ux/M + vy/N)}$$

$$f(x,y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} T(u,v) e^{j2\pi(ux/M + vy/N)}$$

# Image Transforms

**2. Discrete Cosine Transform (DCT)**

- Used in JPEG compression.
- Represents image as cosine basis functions.

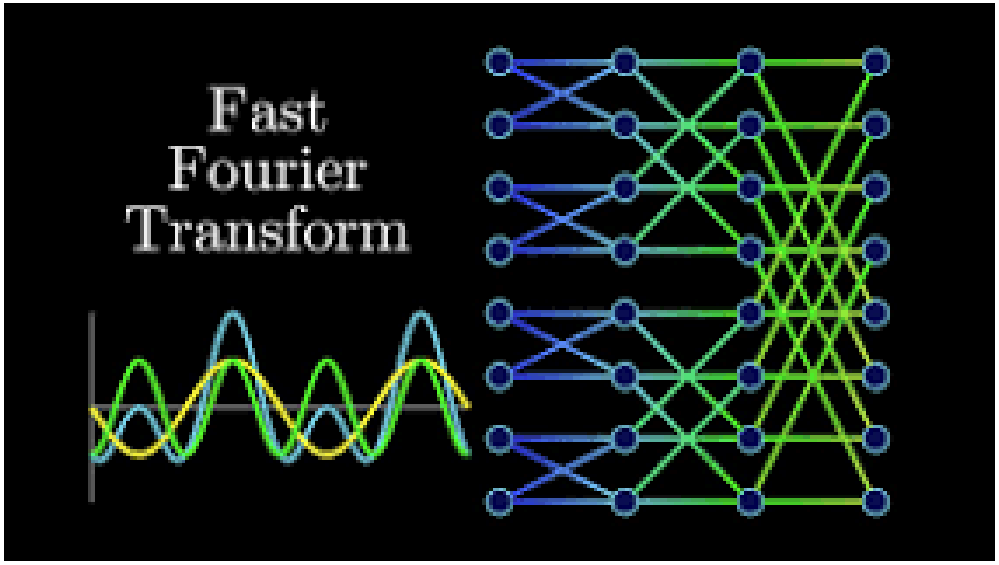
**3. Wavelet Transform**

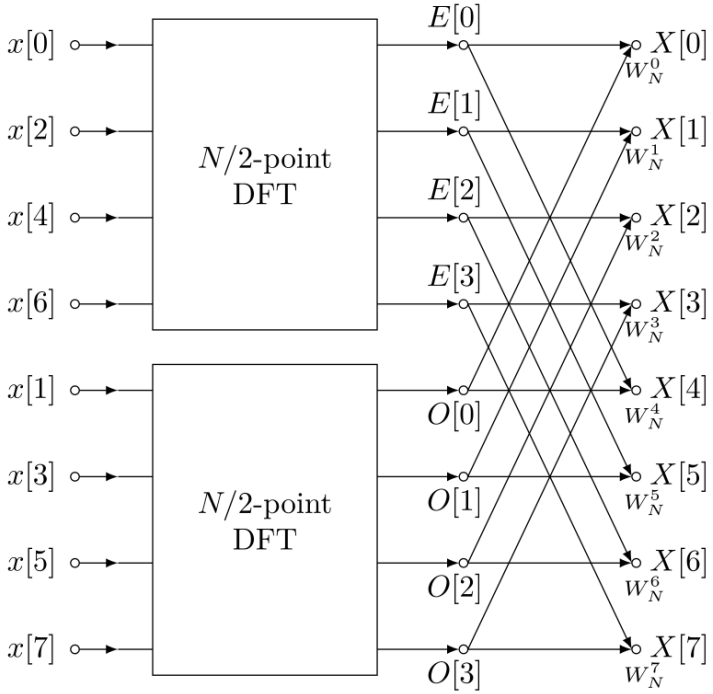
- Multiresolution analysis.
- Used in JPEG2000.

**Applications**

- Image compression (JPEG – DCT, JPEG2000 – Wavelet)
- Medical imaging (Fourier filtering in MRI, CT)
- Object recognition (rotation/scale invariance)
- Video stabilization (geometric transformations)
- Watermarking & encryption

# FFT


Fast
Fourier
Transform


$x[0]$
$x[2]$
$x[4]$
$x[6]$
$x[1]$
$x[3]$
$x[5]$
$x[7]$
$N/2$-point DFT
$N/2$-point DFT
$E[0]$
$E[1]$
$E[2]$
$E[3]$
$O[0]$
$O[1]$
$O[2]$
$O[3]$
$X[0]$
$X[1]$
$X[2]$
$X[3]$
$X[4]$
$X[5]$
$X[6]$
$X[7]$
$W_N^0$
$W_N^1$
$W_N^2$
$W_N^3$
$W_N^4$
$W_N^5$
$W_N^6$
$W_N^7$

# DFT vs IDFT

DFT

$$F(u) = \sum_{x=0}^{M-1} f(x) e^{-j2\pi ux/M} \qquad u = 0, 1, 2, \ldots, M-1$$

DFT pair

and

IDFT

$$f(x) = \frac{1}{M} \sum_{u=0}^{M-1} F(u) e^{j2\pi ux/M} \qquad x = 0, 1, 2, \ldots, M-1$$

# DFT vs IDFT

$$F(u) = F(u + kM) \longleftrightarrow f(x) = f(x + kM)$$

$$f(x) \star h(x) = \sum_{m=0}^{M-1} f(m)h(x - m)$$

# 2-D DFT & IDFT

$$F(u,v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y) e^{-j2\pi(ux/M + vy/N)}$$

$$f(x,y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u,v) e^{j2\pi(ux/M + vy/N)}$$

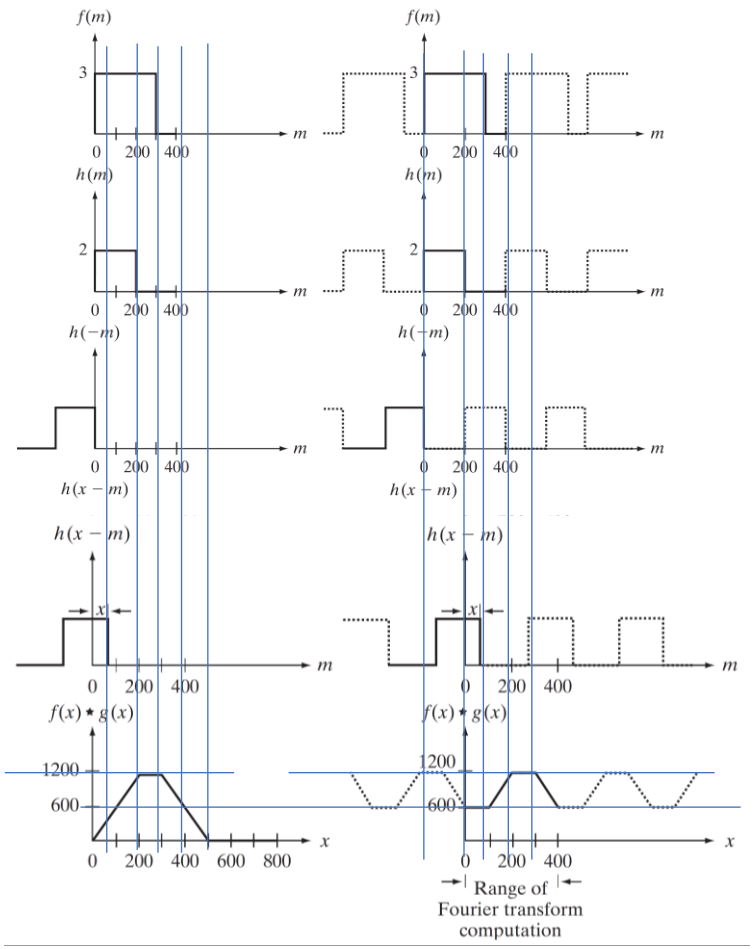
# 2-D Convolution Theorem

$$f(x, y) \star h(x, y) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m, n) h(x - m, y - n)$$

$$f(x, y) \star h(x, y) \Leftrightarrow F(u, v) H(u, v)$$

$$f(x, y) h(x, y) \Leftrightarrow F(u, v) \star H(u, v)$$

# DFT & Convolution Theorem



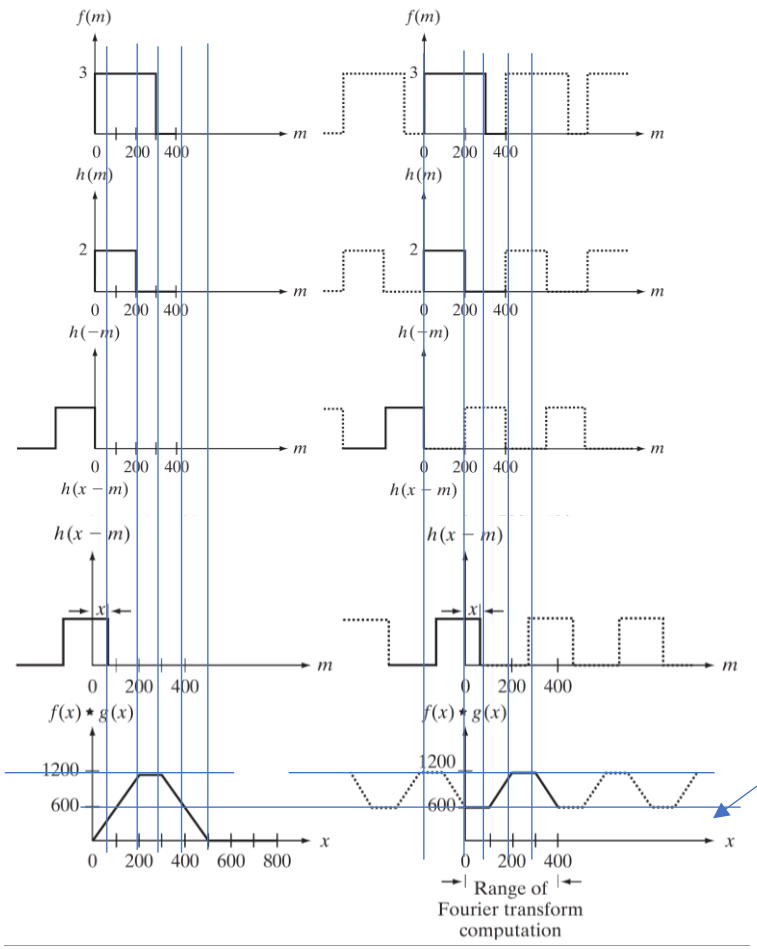

column: convolution of two discrete functions obtained using the approach discussed in Section 3.4.2.The result in (e) is correct. Right column:

Convolution of the same functions, but taking into account the periodicity implied by the DFT.

Note in (j) how data from adjacent periods produce wraparound error, yielding an incorrect convolution result. To obtain the correct result, function padding must be used.

$$f(x) \star h(x) = \sum_{m=0}^{399} f(x)h(x - m)$$

# DFT & Convolution Theorem




A=0-399=400

B=0-399=400

P=>A+B-1

P>=799 (min can be used, append 399 zeros to trailing edge of each function, this process is zero padding)

| | **0** | **2** | **2** |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 3 | 0 | 6 | 6 |
| 3 | 0 | 6 | 6 |

0,0,6, 12, 6,

erroneous 400-point segment

*wraparound error*

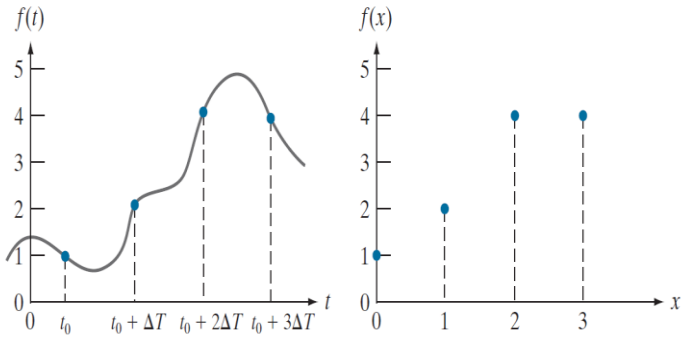
$$f(x) \star h(x) = \sum_{m=0}^{399} f(x)h(x - m)$$

# Example: Sampling using DFT & IDFT

$$F(0) = \sum_{x=0}^{3} f(x) = [f(0) + f(1) + f(2) + f(3)] = 1 + 2 + 4 + 4 = 11$$

$$F(1) = \sum_{x=0}^{3} f(x) e^{-j2\pi(1)x/4} = 1e^{0} + 2e^{-j\pi/2} + 4e^{-j\pi} + 4e^{-j3\pi/2} = -3 + 2j$$

$$f(0) = \frac{1}{4}\sum_{u=0}^{3} F(u) e^{j2\pi u(0)} = \frac{1}{4}\sum_{u=0}^{3} F(u) = \frac{1}{4}[11 - 3 + 2j - 1 - 3 - 2j] = \frac{1}{4}[4] = 1$$



# Example: 2D DFT

Consider a tiny 2×2 grayscale image:

$$f = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$

Objective: Compute the 2D Discrete Fourier Transform (DFT) of this image.

**Step 1: Recall 2D DFT formula**

For an M× $N$ image $f(x, y)$, the DFT $F[u, v]$ is:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi\left(\frac{ux}{N} + \frac{vy}{N}\right)}$$

Where:

- $u, v = 0,1, \ldots, N - 1$
- $j = -1$

For our case, $N = 2$, so $u, v, x, y \in \{0,1\}$.

# Example: 2D DFT

**Step 2: Compute each $F[u, v]$**

- $F[0,0] = \sum_{x=0}^{1} \sum_{y=0}^{1} f(x,y) e^{-j2\pi(0+0)} = 10 + 20 + 30 + 40 = 100$

- $F[0,1] = \sum_{x=0}^{1} \sum_{y=0}^{1} f(x,y) e^{-j2\pi\left(0+\frac{y}{2}\right)} = \sum_{x=0}^{1} \sum_{y=0}^{1} f(x,y) e^{-j2\pi\left(\frac{y}{2}\right)}$

$$= \left[ f[0,0] \times e^{-j2\pi.0} + f[0,1] \times e^{-j2\pi\left(\frac{1}{2}\right)} + f[1,0] \times e^{-j2\pi.0} + f[1,1] \times e^{-j2\pi\left(\frac{1}{2}\right)} \right]$$

$$= [f[0,0] \times 1 + f[0,1] \times (-1) + f[1,0] \times 1 + f[1,1] \times (-1)]$$

$$= [10 \times 1 + 20 \times (-1) + 30 \times 1 + 40 \times (-1)] = -20$$

# Example: 2D DFT

**Step 2: Compute each $F[u, v]$**

- $F[1,0] = \sum_{x=0}^{1} \sum_{y=0}^{1} f(x,y) e^{-j2\pi\left(\frac{x}{2}+0\right)} =$

$$\sum_{x=0}^{1} \sum_{y=0}^{1} f(x,y) e^{-j2\pi\left(\frac{x}{2}\right)}$$

$$= \left[ f[0,0] \times e^{-j2\pi.0} + f[0,1] \times e^{-j2\pi.0} + f[1,0] \times e^{-j2\pi\left(\frac{1}{2}\right)} + f[1,1] \times e^{-j2\pi\left(\frac{1}{2}\right)} \right]$$

$$= [f[0,0] \times 1 + f[0,1] \times 1 + f[1,0] \times (-1) + f[1,1] \times (-1)]$$

$$= [10 \times 1 + 20 \times 1 + 30 \times (-1) + 40 \times (-1)] = -20$$

# Example: 2D DFT

**Step 2: Compute each $F[u, v]$**

***F*[1,1]:**

Calculate exponentials for each $f(x, y)$:

| $(x, y)$ | Exponent calculation | Value |
|---|---|---|
| (0,0) | $e^{-j0} = 1$ | 1 |
| (0,1) | $e^{-j\pi} = -1$ | -1 |
| (1,0) | $e^{-j\pi} = -1$ | -1 |
| (1,1) | $e^{-j2\pi} = 1$ | 1 |

# Example: 2D DFT

**Step 2: Compute each $F[u,v]$**

| $(x,y)$ | Exponent calculation | Value |
|---|---|---|
| (0,0) | $e^{-j0}=1$ | 1 |
| (0,1) | $e^{-j\pi}=-1$ | -1 |
| (1,0) | $e^{-j\pi}=-1$ | -1 |
| (1,1) | $e^{-j2\pi}=1$ | 1 |

$$F[1,1]=\sum_{x=0}^{1}\sum_{y=0}^{1}f(x,y)e^{-j2\pi\left(\frac{x}{2}+\frac{y}{2}\right)}$$

$$=\left[f[0,0]\times e^{0}+f[0,1]\times e^{-j\pi}+f[1,0]\times e^{-j\pi}+f[1,1]\times e^{-j2\pi}\right]$$

$$=\left[f[0,0]\times 1+f[0,1]\times(-1)+f[1,0]\times(-1)+f[1,1]\times 1\right]$$

$$=\left[10\times 1+20\times(-1)+30\times(-1)+40\times 1\right]=0$$

# Example: 2D DFT

**Step 3: Final DFT matrix *F*:**

$$F = \begin{bmatrix} 50 & -10 \\ -20 & 0 \end{bmatrix}$$

**Interpretation**

- $F[0,0]$ is the DC component (average brightness).
- Other components represent different spatial frequencies and phases.
- Negative values are valid complex components (real here because input is real and symmetric).

**Summary:**

| Position | Value | Meaning |
|---|---|---|
| F[0,0] | 50 | Average intensity |
| F[0,1] | -10 | Vertical frequency |
| F[1,0] | -20 | Horizontal frequency |
| F[1,1] | 0 | Diagonal frequency |

# 2D FFT on 2×2 Grayscale Image

Applying the 2D Fast Fourier Transform (FFT) on a 2×2 grayscale image, along with a visual explanation using simple numbers.

$$f = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

The **2D DFT** of a 2×2 image is:

For an $N \times N$ image $f[x, y]$, the DFT $F[u, v]$ is:

$$F(u,v) = \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x,y) e^{-j2\pi\left(\frac{ux}{N}+\frac{vy}{N}\right)}$$

Where:

- $u, v = 0,1,\ldots, N-1$
- $j = -1$

For our case, $N = 2$, so $u, v, x, y \in \{0,1\}$.

$$F(u,v) = \sum_{x=0}^{1} \sum_{y=0}^{1} f(x,y) e^{-j2\pi\left(\frac{ux}{N}+\frac{vy}{N}\right)}$$

We'll compute all $F(u,v)\ for\ u, v = 0\ or\ 1$.

# 2D FFT on 2×2 Grayscale Image

**Step 1: Compute *F*(0,0)**

$$F(0,0) = 1 + 2 + 3 + 4 = 10$$

**Step 2: Compute *F*(0,1)**

$$F(0,1) = (1)(1) + (2)(-1) + (3)(1) + (4)(-1) = -2$$

**Step 3: Compute *F*(1,0)**

$$F(1,0) = (1)(1) + (2)(1) + (3)(-1) + (4)(-1) = -4$$

**Step 4: Compute *F*(1,1)**

$$F(1,1) = (1)(1) + (2)(-1) + (3)(-1) + (4)(1) = 0$$

# 2D FFT on 2×2 Grayscale Image

**Final FFT Output Matrix:**

$$F = \begin{bmatrix} 10 & -2 \\ -4 & 0 \end{bmatrix}$$

**Interpretation:**

- **F(0,0)** = 10 is the **DC component** (average brightness).

- Others are **frequency components** showing contrast, edges, and patterns.

# Image DFT and IDFT

| Step | Result |
| --- | --- |
| **DFT** | Converts image to frequency domain |
| **IDFT** | Reconstructs the original image |
| **Accuracy** | Perfect match (no loss) |
| **Limitation** | Small size shown; large images use FFT algorithms |

# Example: Image DFT and IDFT

**Step 1: Given a 2×2 image**

$$f(x,y) = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

We'll compute:

- **2D DFT** to get frequency domain representation.
- **2D Inverse DFT (IDFT)** to reconstruct the original image.

**Step 2: Formula for 2D DFT**

For an $N \times N$ image $f[x,y]$, the DFT $F[u,v]$ is:

$$F(u,v) = \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} I(x,y) e^{-j2\pi\left(\frac{ux}{N}+\frac{vy}{N}\right)}$$

Where: $u, v = 0,1,\ldots,N-1$

For our case, $N = 2$, so $u, v, x, y \in \{0,1\}$.

The complex exponential $e^{-\frac{j2\pi k}{2}}$ cycles between $e^0 = 1$ & $e^{-j\pi} = -1$.

# Example: Image DFT and IDFT

**Step 3: Compute DFT coefficients *F*(*u*,*v*)**

Let's compute all 4 frequency components:

$$f(x,y) = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

***F*(0,0):** $F(0,0) = 1 + 2 + 3 + 4 = 10$

***F*(0,1):** $F(0,1) = 1(1) + 2(-1) + 3(1) + 4(-1) = -2$

***F*(1,0):** $F(1,0) = 1(1) + 2(1) + 3(-1) + 4(-1) = -4$

***F*(1,1):** $F(1,1) = 1(1) + 2(-1) + 3(-1) + 4(1) = 0$

**DFT Output:**

$$F(u,v) = \begin{bmatrix} 10 & -2 \\ -4 & 0 \end{bmatrix}$$

# Example: Image DFT and IDFT

**Step 4: Inverse DFT (IDFT)**

IDFT formula:

$$f(x,y) = \frac{1}{M \times N} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u,v) e^{-j2\pi\left(\frac{ux}{N}+\frac{vy}{N}\right)}$$

Here, $N$=2, so divide by 4 and use $e^{j0} = 1$ and $e^{j\pi} = -1$.

# Example: Image DFT and IDFT

**Step 5: Reconstruct *f*(*x*,*y*)**

$$f(0,0) = \frac{10 + (-2)(1) + (-4)(1) + 0}{4} = 1$$

$$f(0,1) = \frac{10 + (-2)(-1) + (-4)(1) + 0}{4} = 2$$

$$f(1,0) = \frac{10 + (-2)(1) + (-4)(-1) + 0}{4} = 3$$

$$f(1,1) = \frac{10 + (-2)(-1) + (-4)(-1) + 0}{4} = 4$$

**Final Reconstructed Image:**

$$f(x,y) = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

**Perfectly matches the original.**

# Thank You

# Lecture 4.3

## Image Transforms

# Walsh/Fast Walsh (Walsh–Hadamard) Transform, Karhunen and Loeve Transform (KLT)

Instructor Name:

**Professor Dr. Jahangir Alam**

**Dept. of CSE, BAUST**

# Walsh Transform for Image Transform

- The Walsh/Fast Walsh (Walsh–Hadamard) transform (FWHT) is a useful, fast, and purely real orthogonal transform that—like the Fourier/DCT family—can be applied to images.
- FWHT expands a signal (or image) into a basis of piecewise-constant orthogonal functions that take values +1 or −1 (no sines/cosines).
- Basis functions are rectangular "waveforms" that change sign a certain number of times — the number of sign changes is called sequency (analogue of frequency).
- The transform is very efficient (no multiplications required in the "butterfly" implementation) and is popular in integer/hardware settings.
- The butterfly structure is a graphical way to represent how data elements are combined and split recursively during the transform — similar to the FFT but FWHT uses only additions and subtractions (no multiplications).

# Walsh Transform for Image Transform

**When to use it?**

- Fast operations on devices where multiplications are expensive.
- Image compression and watermarking research (less energy compaction than DCT often, but simpler hardware).
- Pattern recognition and binary-feature extraction.
- Situations needing an orthogonal ±1 basis or fast integer-friendly transforms.

**Applications (image)**

- Lightweight image compression prototypes.
- Watermarking and feature extraction.
- Image fusion, recognition, fast correlation.
- Hardware-friendly transforms in embedded vision.

# Walsh Transform for Image Transform

**Mathematical definition (1D)**

- For $N = 2^m$ samples $x[n]$, the Walsh–Hadamard transform (WHT) produces coefficients $X[k]$:

$$X[k] = \sum_{n=0}^{N-1} x[n] H_{k,n}$$

- where $H$ is the $N \times N$ Hadamard matrix (entries ±1). Common normalization choices:
- Unnormalized forward: above. Inverse is $x[n] = \frac{1}{N} \sum_k X[k]\, H_{k,n}$.
- Symmetric orthonormal: multiply by $\frac{1}{\sqrt{N}}$ for both forward and inverse.

# Walsh Transform for Image Transform

- For an image $f(x, y)$ of size $N \times N$:

$$F(u, v) = W_N \cdot f(x, y) \cdot W_N^T$$

- where $W_N$ is the Walsh matrix, derived from the Hadamard matrix $H_N$ but reordered based on sequency.

$$W_N = P . H_N$$

- Here, $P$ is a permutation matrix that sorts rows of $W_N$ in increasing order of sequency.

# Walsh Transform for Image Transform

**Sequency Concept**

Sequency = Number of zero crossings per row (or column) in the basis function.

Example (for 4×4 Walsh basis):

This ordering makes the transform suitable for image frequency analysis similar to DCT.

| Row | Pattern | Sequency |
|---|---|---|
| 1 | + + + + | 0 |
| 2 | + + − − | 1 |
| 3 | + − − + | 2 |
| 4 | + − + − | 3 |

# Walsh Transform for Image Transform

| Application | Purpose |
| --- | --- |
| **Image Compression** | Similar to DCT; energy compaction in lower sequency coefficients |
| **Feature Extraction** | Useful for texture and pattern recognition |
| **Face Recognition** | Used in early pattern matching systems |
| **Watermarking** | Embedding hidden signals |
| **Optical Computing** | Easy implementation with digital hardware |

# Walsh Transform for Image Transform

**Mathematical definition (1D)**

- Hadamard matrix example ($N = 4$, Walsh–Paley ordering):

$$H_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

- Sequency ordering (Walsh ordering) reorders rows of H by number of sign changes.

# Walsh Transform for Image Transform

Input Image (Spatial Domain)

↓

Walsh Transform (Walsh Matrix)

↓

Walsh Coefficients (Frequency Domain)

↓

Quantization (Lossy Compression)

↓

Inverse Walsh Transform

↓

Reconstructed Image

# Walsh Transform for Image Transform

**2D Walsh–Hadamard for images**

- 2D-WHT is separable:
- Apply 1D-WHT to each row.
- Apply 1D-WHT to each column (or vice versa).
- Inverse: same operations, then scale by $\frac{1}{N}$ (or $\frac{1}{\sqrt{N}}$ depending on normalization). Complexity: $O(N\ logN))$ per 1D vector, so overall $O\left(N_x N_y\ logN\right)$.

# Walsh Transform for Image Transform

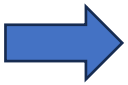
- For a 2x2 image, the Walsh matrix *W* is:

$$W = \frac{1}{2}\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

- Since our image is $2 \times 2$, we flatten it into a $4 \times 1$ vector, apply *W*, then reshape back.

**Step 1:** Define a is $2 \times 2$ image matrix *f*

$$f = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$



$$X = \begin{bmatrix} 10 \\ 20 \\ 30 \\ 40 \end{bmatrix}$$

# Walsh Transform for Image Transform

$$W = \frac{1}{2}\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}, f = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \quad \Rightarrow \quad X = \begin{bmatrix} 10 \\ 20 \\ 30 \\ 40 \end{bmatrix}$$

Step 3: Apply Walsh transform, $y = W \times x$

$$y1 = \frac{1}{2}(10 + 20 + 30 + 40) = 12 \times 100 = 50$$

$$y2 = \frac{1}{2}(10 - 20 + 30 - 40) = 12 \times (-20) = -10$$

$$y3 = \frac{1}{2}(10 + 20 - 30 - 40) = 12 \times (-40) = -20$$

$$y4 = \frac{1}{2}(10 - 20 - 30 + 40) = 12 \times 0 = 0$$

$$Y = \begin{bmatrix} 50 & -10 \\ -20 & 0 \end{bmatrix}$$

# Hadamard Transform for Image Transform

For a 1D signal $f(x)$ of length $N$:

$$F(u) = H_N \cdot f(x)$$

where $H_N$ is the Hadamard matrix of order $N$, recursively defined as:

$$H_1 = [1]$$

$$H_2 = \begin{bmatrix} H_N & H_N \\ H_N & -H_N \end{bmatrix}$$

For a 2D image $f(x, y)$:

$$F = H_N \cdot f \cdot H_N^T$$

# Hadamard Transform for Image Transform

Input Image (Spatial Domain)

↓

Hadamard Transform

↓

Hadamard Coefficients (Frequency Domain)

↓

Quantization (Lossy step)

↓

Inverse Hadamard Transform

↓

Reconstructed Image

# Hadamard Transform for Image Transform

**Applications**

| Use Case | Description |
|---|---|
| **Compression** | Simple transform coding (like DCT) |
| **Feature Extraction** | For face recognition or texture analysis |
| **Watermarking** | Embedding hidden data in frequency coefficients |
| **Optical Imaging** | Used in compressive sensing systems |

# Hadamard Transform for Image Transform

**Hadamard Transform basics:**

- Hadamard matrix $H$ is an orthogonal matrix with entries $+1$ and $-1$.
- For size $2\times2$, the Hadamard matrix is:

$$H_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

- The 2D Hadamard transform of image $I$ is computed as:

$$Y = H \times I \times H$$

Step 1: Define a $2\times2$ image matrix $I$

$$I = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$

# Hadamard Transform for Image Transform

Step 1: Define a 2×2 image matrix $I$

$$I = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$

$$H_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Step 2: Compute $H \times I$

$$H \times I = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \times \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$

$$= \begin{bmatrix} 1 \times 10 + 1 \times 30 & 1 \times 20 + 1 \times 40 \\ 1 \times 10 + (-1) \times 30 & 1 \times 20 + (-1) \times 40 \end{bmatrix} = \begin{bmatrix} 40 & 60 \\ -20 & -20 \end{bmatrix}$$

Step 3: Compute $Y=(H \times I) \times H$

$$Y = \begin{bmatrix} 40 & 60 \\ -20 & -20 \end{bmatrix} \times \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$= \begin{bmatrix} 40 \times 1 + 60 \times 1 & 40 \times 1 + 60 \times (-1) \\ -20 \times 1 + (-20) \times 1 & -20 \times 1 + (-20) \times (-1) \end{bmatrix}$$

$$= \begin{bmatrix} 100 & -20 \\ -40 & 0 \end{bmatrix}$$

# Hadamard Transform for Image Transform

Step 4: Normalize the result

The Hadamard transform is often normalized by dividing by the size $N$=2:

$$Y_{normalized} = \frac{1}{N} Y = \frac{1}{2} \times \begin{bmatrix} 100 & -20 \\ -40 & 0 \end{bmatrix} = \begin{bmatrix} 50 & -10 \\ -20 & 0 \end{bmatrix}$$

Final Hadamard Transform matrix:

$$\begin{bmatrix} 50 & -10 \\ -20 & 0 \end{bmatrix}$$

Interpretation:

- The value 50 is the DC component (average brightness).
- Other coefficients correspond to spatial frequency components of the image.

# Walsh-Hadamard Algorithm (WHT)

**Algorithm Steps (2D Walsh–Hadamard Transform)**

**Step 1: Input**

Take a grayscale image $f(x, y)$ of size $N \times N$.

$$F(u, v) = H_N \cdot f(x, y) \cdot H_N^T$$

**Step 2: Generate Hadamard Matrix**

Construct the Hadamard matrix $H_N$ recursively.

**Step 3: Apply Transform**

Compute:

$$F(u, v) = H_N \cdot f(x, y) \cdot H_N^T$$

This gives the frequency-domain coefficients (Walsh–Hadamard coefficients).

**Step 4: Quantization (optional)**

Reduce precision of coefficients for lossy compression.

**Step 5: Inverse Transform**

Reconstruct the image:

$$f(x, y) = \frac{1}{N^2} H_N^T.$$

# Walsh-Hadamard Algorithm (WHT)

## Applications of Walsh–Hadamard Transform

| Application Area | Use Case |
|---|---|
| Image Compression | Replace DCT (simpler computation, integer-only) |
| Texture Analysis | Identify spatial frequency patterns |
| Image Matching / Classification | Extract WHT features as descriptors |
| Watermarking / Steganography | Embed signals in transform coefficients |
| Medical Image Processing | Compact representation for CT/MRI data |
| Face Recognition (Early Models) | Use WHT features instead of PCA/DCT |

# Walsh-Hadamard Algorithm (WHT)

**Walsh–Hadamard Transform Pipeline (Image Analysis)**

Input Image (Spatial Domain)

↓

Walsh–Hadamard Transform

↓

WHT Coefficients (Frequency Domain)

↓

Feature Extraction / Compression / Segmentation

↓

Inverse WHT (Reconstruction)

↓

Output / Analysis Result

# Walsh-Hadamard Algorithm (WHT)

The WHT is a non-sinusoidal, orthogonal transform using only +1 and -1 values. It's useful for fast image processing and analysis.

**Step 1:** Small Image (4×4)

Consider a simple 4×4 grayscale image matrix *f*:

$$f = \begin{bmatrix} 12 & 15 & 10 & 13 \\ 14 & 16 & 11 & 12 \\ 13 & 14 & 12 & 15 \\ 16 & 18 & 14 & 17 \end{bmatrix}$$

**Step 2:** Walsh-Hadamard Matrix H4

The 4×4 Hadamard matrix is:

$$H_4 = \frac{1}{2}\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

# Walsh-Hadamard Algorithm (WHT)

Step 3: Compute WHT: $W=H4\times I\times H4$
This is a 2D transform: multiply image from left and right by the Hadamard matrix.

$$f=\begin{bmatrix}12 & 15 & 10 & 13\\ 14 & 16 & 11 & 12\\ 13 & 14 & 12 & 15\\ 16 & 18 & 14 & 17\end{bmatrix}$$

Step 3a: Calculate intermediate matrix $M=H4\times I$
Multiply each element of $H4$ row by $I$ column and sum.

$$H_4=\frac{1}{2}\begin{bmatrix}1 & 1 & 1 & 1\\ 1 & -1 & 1 & -1\\ 1 & 1 & -1 & -1\\ 1 & -1 & -1 & 1\end{bmatrix}$$

Calculate $M_{1,1}$ (first row, first column):

$$M_{1,1}=\frac{1}{2}(1*12+1*14+1*13+1*16)=27.5$$

Similarly, $M(1,2)$ (first row, second column):

$$M_{1,2}=\frac{1}{2}(1*15+1*16+1*14+1*18)=31.5$$

# Walsh-Hadamard Algorithm (WHT)

$$f = \begin{bmatrix} 12 & 15 & 10 & 13 \\ 14 & 16 & 11 & 12 \\ 13 & 14 & 12 & 15 \\ 16 & 18 & 14 & 17 \end{bmatrix}$$

$$H_4 = \frac{1}{2}\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

Continue for all elements of *M*.

For brevity, I'll compute the full *M* matrix values:

| Row/ Col | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 27.5 | 31.5 | 23.25 | 25.75 |
| 2 | -0.5 | -0.5 | -0.25 | -0.25 |
| 3 | 5.5 | 6.5 | 2.25 | 3.75 |
| 4 | -2.5 | -2.5 | -1.25 | -1.25 |

# Walsh-Hadamard Algorithm (WHT)

$$f = \begin{bmatrix} 12 & 15 & 10 & 13 \\ 14 & 16 & 11 & 12 \\ 13 & 14 & 12 & 15 \\ 16 & 18 & 14 & 17 \end{bmatrix}$$

$$H_4 = \frac{1}{2}\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

Step 3b: Calculate $W=M\times H4$

Multiply $M$ by $H4$ from the right.

Calculate $W_{1,1}$:

$$W_{1,1} = \frac{1}{2} \times \sum_{k=1}^{4} M_{1,k} \times H_{k,1}$$

$$= \frac{1}{2}(27.5 * 1 + 31.5 * 1 + 23.25 * 1 + 25.75 * 1)$$

$$= \frac{1}{2}(27.5 + 31.5 + 23.25 + 25.75) = 21 \times 108$$

$$= 54$$

Similarly for other elements.

# Walsh-Hadamard Algorithm (WHT)

**Step 4: Interpretation of WHT Coefficients**

•The first element *W1,1* is the DC component (average brightness).

•Other elements represent frequency components (patterns) in the image.

•Large coefficients correspond to important features or edges.

**Summary:**

| Matrix | Size | Role |
|---|---|---|
| I | 4×4 | Original image |
| H4 | 4×4 | Walsh-Hadamard basis |
| W | 4×4 | Transformed image (frequency) |

**Why WHT for Image Analysis?**

•Fast computation (only additions and subtractions).

•Useful for pattern recognition.

•Captures image details in orthogonal basis.

# Karhunen and Loeve Transform (KLT)

What is KLT?

- KLT is an optimal transform for decorrelating data.
- Finds eigenvectors of the covariance matrix of the image data.
- Used for dimensionality reduction, compression, & feature extraction.

The KLT (also called **Hotelling Transform or PCA transform**) is a *statistically optimal linear transform* for energy compaction.

It's foundational in image and signal processing — the mathematical "ancestor" of PCA, DCT, and other transforms.

**Applications:**

- Image **compression** (energy compaction)
- **Face recognition** (Eigenfaces → PCA)
- **Feature extraction** and **pattern recognition**
- **Denoising** and **dimensionality reduction**

# Karhunen and Loeve Transform (KLT)

**Steps to Compute KLT for Image Transform**

1. **Prepare data**

   Flatten or take small image blocks (e.g. 8×8). Let's say each block is a vector **X**.

$$\boldsymbol{X} = [x_1, x_2, x_3, \dots, x_N]^T$$

1. **Compute mean vector**

$$\bar{X} = \frac{1}{M}\sum_{i=1}^{M} X_i$$

1. **Compute covariance matrix**

$$C_x = E[(X_i - \bar{X})(X_i - \bar{X})^T] = \frac{1}{M}\sum_{i=1}^{M}(X_i - \bar{X})(X_i - \bar{X})^T$$

# Karhunen and Loeve Transform (KLT)

4. **Compute eigenvectors and eigenvalues**

$$C_x \mathrm{A} = \mathrm{A}\Lambda \Rightarrow A^T C_x \mathrm{A} = \Lambda$$

where A is the **orthonormal matrix of eigenvectors of** $C_x$, and $\Lambda$ is a **diagonal matrix** of eigenvalues.

6. **KLT Transform is defined as,**

$$Y_i = A^T (X_i - \bar{X})$$

8. **(Optional)** Keep only the first $k$ largest eigenvalues → **compression**.

9. **Inverse transform**

$$X_i = A y_i + \bar{X}$$

# Karhunen and Loeve Transform (KLT)

**Example: 2D Dataset (2 Features, 3 Samples)**

Let's consider 3 data points in 2D space (each column is a sample):

$$X = \begin{bmatrix} 2 & 4 & 6 \\ 1 & 3 & 5 \end{bmatrix}$$

Each column is a sample vector:

- $x1 = [21], x2 = [43], x3 = [65]$

**Step 1: Compute the Mean Vector**

$$\mu = (x1 + x2 + x3)/3 = 13[2 + 4 + 61 + 3 + 5] = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

**Step 2: Mean-Center the Data**

Subtract mean from each column:

$$Xc = X - \mu = [2 - 44 - 46 - 41 - 33 - 35 - 3]$$

$$= \begin{bmatrix} -2 & 0 & 2 \\ -2 & 0 & 2 \end{bmatrix}$$

# Karhunen and Loeve Transform (KLT)

**Step 3: Covariance Matrix**

Let Xc be 2×3. Covariance matrix = $C = \frac{1}{n} X_c X_c^T$

$$C = \frac{1}{3}[-202 - 202][-2 - 20022] = \begin{bmatrix} 2.67 & 2.67 \\ 2.67 & 2.67 \end{bmatrix}$$

**Step 4: Eigenvalues and Eigenvectors of Covariance Matrix**

Let's find eigenvalues $\lambda$ of:

$\begin{bmatrix} 2.67 & 2.67 \\ 2.67 & 2.67 \end{bmatrix}$ ⇒Characteristic equation: $\begin{bmatrix} 2.67 - \lambda & 2.67 \\ 2.67 & 2.67 - \lambda \end{bmatrix} = 0$

$$(2.67 - \lambda)^2 - 2.67^2 = 0 \Rightarrow \lambda^2 - 5.34\lambda = 0$$

⇒eigenvalues: λ1=5.34, λ2=0

# Karhunen and Loeve Transform (KLT)

Now for eigenvectors:

- For $\lambda$=5.34, solve $(C - \lambda I)v = 0$

  You get: $v1 = \begin{bmatrix}1\\1\end{bmatrix}$ (or any scalar multiple)

- For $\lambda = 0, v2 = \begin{bmatrix}1\\-1\end{bmatrix}$

We normalize:

$$v1 = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix}, v2 = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\-1\end{bmatrix}$$

**Step 5: KLT Transformation**

Let $V = [v1\ v2]$ = eigenvector matrix: $v2 = \frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\1 & -1\end{bmatrix}$

Now apply KLT:

$Y = V^T X_c \Rightarrow$project data onto uncorrelated axes

$$Y = V^T X_c = \frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\1 & -1\end{bmatrix}^T \begin{bmatrix}-2 & 0 & 2\\-2 & 0 & 2\end{bmatrix} = \frac{1}{\sqrt{2}}\begin{bmatrix}-4 & 0 & 4\\0 & 0 & 0\end{bmatrix}$$

# Karhunen and Loeve Transform (KLT)

**Result**

- Data decorrelated (transformed into uncorrelated variables).
- One eigenvalue is 0 → data lies along a **single direction** (the first component).
- This reveals **dimensionality reduction** potential (PCA).
- The 1st principal component (first row of Y) contains the significant variation.
- The 2nd component is 0 for all samples, showing data lies along a single line.
- KLT decorrelates data into uncorrelated components (principal components).

# Karhunen and Loeve Transform (KLT)

**What You've Learned**

| Step | Concept |
|---|---|
| 1 | Compute mean |
| 2 | Center data |
| 3 | Compute covariance matrix |
| 4 | Find eigenvalues and eigenvectors |
| 5 | Transform data to KLT space |

# 2-D KLT for Image Processing

**Step 1:** Given a small image (2×2 pixels for simplicity)

$$I = \begin{bmatrix} 52 & 55 \\ 53 & 54 \end{bmatrix}$$

**Step 2:** Vectorize the image

Flatten $I$ into a vector $x$:

$$x = \begin{bmatrix} 52 \\ 55 \\ 53 \\ 54 \end{bmatrix}$$

**Step 3:** Compute mean vector $\mu$

$$\mu = \frac{1}{4}(52 + 55 + 53 + 54) = 53.5$$

**Step 4:** Subtract mean to get zero-mean vector $xc$

$$x_c = x - \mu = \begin{bmatrix} 52 - 53.5 \\ 55 - 53.5 \\ 53 - 53.5 \\ 54 - 53.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ 1.5 \\ -0.5 \\ 0.5 \end{bmatrix}$$

# 2-D KLT for Image Processing

Step 5: Form data matrix *Xc*

Assuming this is a single sample, the covariance matrix is computed from *xc* as:

$$C = x_c \times x_c^T = \begin{bmatrix} -1.5 \\ 1.5 \\ -0.5 \\ 0.5 \end{bmatrix} [-1.5 \quad 1.5 \quad -0.5 \quad 0.5]$$

$$= \begin{bmatrix} 2.25 & -2.25 & 0.75 & -0.75 \\ -2.25 & 2.25 & -0.75 & 0.75 \\ 0.75 & -0.75 & 0.25 & -0.25 \\ -0.75 & 0.75 & -0.25 & 0.25 \end{bmatrix}$$

# 2-D KLT for Image Processing

Step 6: Compute eigenvalues and eigenvectors of $C$
Solve $det(C - \lambda I) = 0$.

For simplicity, approximate eigenvalues and eigenvectors:

- Principal eigenvector corresponds to direction of greatest variance.
- Here, the eigenvector might look like:

$$v1 = \frac{1}{2}[1, -1, 1, -1]^T$$

Step 7: Project the zero-mean data on eigenvectors

$$y = v_c^T x_c = \frac{1}{2}[1, -1, 1, -1] \cdot \begin{bmatrix} -1.5 \\ 1.5 \\ -0.5 \\ 0.5 \end{bmatrix} = -2$$

# 2-D KLT for Image Processing

Step 8: Reconstruction (optional)

Reconstruct approx. image:

$$\hat{x}_c = yv_1 = -2 \times \frac{1}{2}[1,-1,1,-1]^T = [-1,1,-1,1]^T$$

Add mean back:

$$\hat{x} = \hat{x}_c + \mu = \begin{bmatrix} 52.5 \\ 54.5 \\ 52.5 \\ 54.5 \end{bmatrix}$$

# Thank You